

Computer Animation

From storyboards and key frames to character rigging, periodic motion and real time OpenGL animation. How pictures are made to move, smoothly and believably, one frame at a time.

Animation Sequences

Key Frames & Tweening

Character Rigging

Periodic Motion

OpenGL Animation

Module Roadmap

Seven topics that take us from planning a scene to animating it on screen

Part A · Concepts and Techniques

- Design of animation sequences
- Traditional animation techniques
- General computer animation functions
- Computer animation languages

Part B · Motion and Programming

- Character animation and kinematics
- Periodic motion with sine and cosine
- OpenGL animation procedures
- A complete bouncing object example

CORE IDEA

Animation is the rapid display of slightly different frames. Show 24 or more frames per second and the eye fuses them into continuous motion.

What is Computer Animation?

Generating a series of frames where objects change position, shape or colour over time

Computer animation refers to any time varying sequence of pictures produced with a computer. Each still picture is a **frame**. When frames are displayed quickly in order, **persistence of vision** makes the change look like smooth motion.

24 fps

Film standard frame rate

30 to 60 fps

Video and interactive graphics

Frame

One still image in the sequence

Real time

Frames generated as they are shown



Frame by frame

Each frame is fully computed and stored, then played back, as in films.



Real time

Frames are drawn on the fly, as in games and simulations.

Design of Animation Sequences

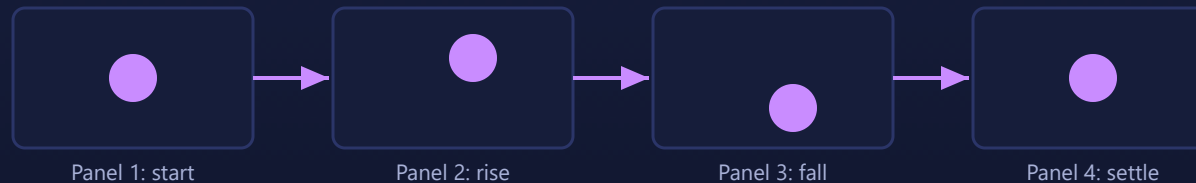
A large animation is planned in careful stages before any frame is rendered

1 Storyboard layout. An outline of the action, drawn as a sequence of sketch panels that define the motion sequence.

2 Object definitions. Each participant is described by its shape, and by the movements it will perform.

3 Key frame specification. Detailed drawings of the scene at certain critical moments in time.

4 In between generation. The frames that fill the gaps between one key frame and the next.



A storyboard breaks the action into panels that define the flow of the sequence.

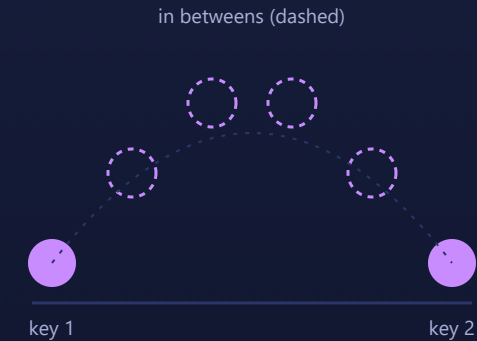
Key Frames and In Betweens

Skilled artists draw the key poses; the gaps are filled to complete the motion

- A **key frame** is a detailed drawing at a critical moment, such as the extreme of a movement.
- **In betweens** are the intermediate frames placed between two key frames.
- The number of in betweens depends on the display frame rate and the time between keys.
- Moving 1 second at 24 fps between two keys needs about 22 in betweens.

INTERPOLATION

In betweens are produced by interpolating positions and shapes between the two surrounding key frames.



Solid circles are key frames; dashed circles are interpolated in betweens.

Steps of Producing an Animation

The full production pipeline, from concept to the finished sequence

- 1 Story and storyboard.** Decide the action and lay it out as sketch panels.
- 2 Model the objects.** Define shapes, colours and the movements each object makes.
- 3 Set key frames.** Specify the scene at the critical instants of the motion.
- 4 Generate in between.** Interpolate all intermediate frames.
- 5 Render.** Apply lighting, texture and surface detail to each frame.
- 6 Record and play back.** Store the frames and display them at the chosen frame rate.

Traditional Animation Techniques

Principles from hand drawn cartoons still guide computer animation today

Long before computers, animators developed craft techniques for making motion feel alive. Many of these ideas are now built into computer animation tools.



Cel animation

Characters drawn on clear celluloid sheets, layered over a fixed painted background.



Key framing

A lead artist draws the extreme poses; assistants fill the frames between them.



Tweening

Producing the in between frames, by hand traditionally, automatically on a computer.



Squash and stretch

Deform an object to suggest weight, speed and elasticity while keeping its volume.



Timing

The number of frames per action controls how fast or slow a motion appears.

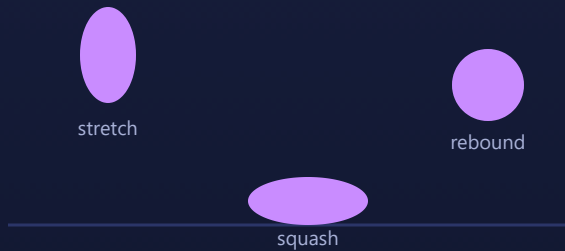


Ease in and ease out

Motion speeds up and slows down gently rather than starting and stopping abruptly.

Squash, Stretch and Timing

Two of the most important principles for believable motion



The ball stretches as it falls fast, squashes on impact, then returns to shape.

Timing

Timing is set by how many frames a movement occupies.

- **Few frames** for a distance means a fast, snappy motion.
- **Many frames** for the same distance means slow, heavy motion.
- Timing conveys weight, mood and the physical scale of an object.

VOLUME RULE

When an object squashes or stretches, its total volume should stay roughly constant, or it looks like it is growing or shrinking.

General Computer Animation Functions

The common capabilities that animation software provides



Object manipulation

Create, position, move, rotate and scale objects over time.



Camera motion

Pan, zoom, tilt and dolly the viewpoint through the scene.



Generating in between

Automatically interpolate frames between specified key frames.



Morphing

Smoothly transform one object shape into a different one.



Lighting and rendering

Set and change light sources, materials and surface detail per frame.



Motion paths

Move objects along defined curves such as splines.

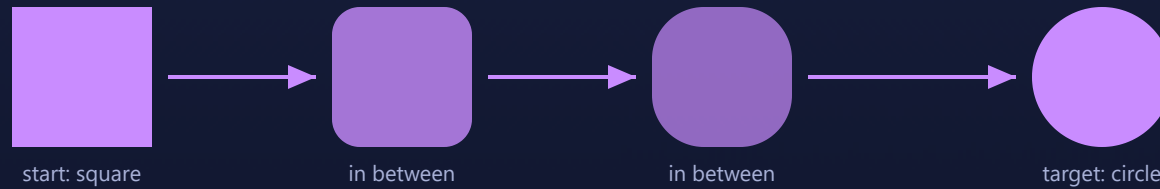
STORING MOTION

Software stores object descriptions and a set of transformation parameters that change from frame to frame, rather than every full picture.

Morphing

Transforming one object shape into another through in between shapes

Morphing (metamorphosis) gradually reshapes a starting object into a target object. The system matches features of the two shapes, then interpolates vertex positions, edges and colours across a run of in between frames.



Corner vertices are interpolated over several frames to turn a square into a circle.

FEATURE MATCHING

When the two shapes have different numbers of vertices or edges, extra points are added so the two outlines can be matched before interpolation.

Camera Motion

Moving the viewpoint is as expressive as moving the objects



The camera has its own position and view direction that can animate over time.

Move	What it does
Pan	Rotate the camera left or right
Tilt	Rotate the camera up or down
Zoom	Change the field of view, wider or narrower
Dolly	Move the camera toward or away from the scene
Track	Move the camera sideways with the action

Computer Animation Languages

Three families of systems for specifying how a scene animates

System type	How you specify motion	Best for
Key frame systems	You supply key frames; the system generates the in betweens	Traditional style character and object motion
Parameterized systems	You set parameters such as speed, degrees of freedom and motion rules	Repeated or rule driven motion
Scripting systems	You write a script in an animation language describing objects and actions	Complex, programmable, reusable sequences



Key frame

Object attributes are stored at key frames and interpolated between them.



Parameterized

Motion characteristics are given as parameters, for example joint limits and speeds.



Scripting

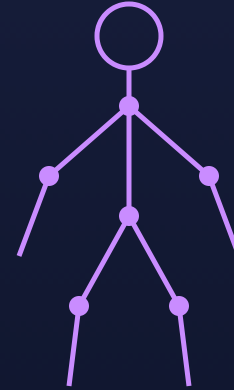
A user language lets you describe object motion and interaction as code.

Character Animation

Animating figures built from connected, jointed parts

Characters such as people and animals are modelled as **articulated figures**: a set of rigid segments connected at **joints**. The internal frame of joints and bones is the **skeleton**.

- **Articulated figure**: rigid links joined at movable joints.
- **Skeleton**: the hierarchy of bones that drives the shape.
- **Rigging**: attaching the visible surface (skin) to the skeleton.
- **Skinning**: how skin deforms as joints rotate.



A skeleton of bones (lines) and joints (dots) drives the character.

Forward vs Inverse Kinematics

Two ways to compute how a jointed figure moves

Forward Kinematics

- You set the **joint angles** directly.
- The system computes where the end of the chain (the hand or foot) ends up.
- Simple and predictable to calculate.
- Harder to make the hand reach an exact target.

vs

Inverse Kinematics

- You set the **target position** of the end effector.
- The system solves for the joint angles that reach it.
- Natural for tasks like touching a point or planting a foot.
- Can have many solutions, so it needs constraints.

Forward: set angles, find the tip



Inverse: set the target, find angles



Forward starts from the joints; inverse starts from the target and works back.

Periodic Motion

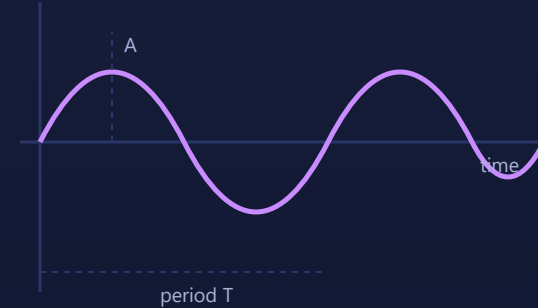
Sine and cosine functions produce smooth, repeating movement

Many natural motions repeat: a swinging pendulum, a bobbing float, a beating wing. These are modelled with the **sine** and **cosine** functions, which oscillate smoothly between fixed limits.

```
// oscillate a position over time
y = A * sin(w * t + phi);

// A    = amplitude (max displacement)
// w    = angular frequency = 2*PI*f
// t    = time
// phi  = phase (starting offset)
```

- **Amplitude** sets how far the object swings.
- **Frequency** sets how fast it repeats.
- **Phase** shifts the start of the cycle.

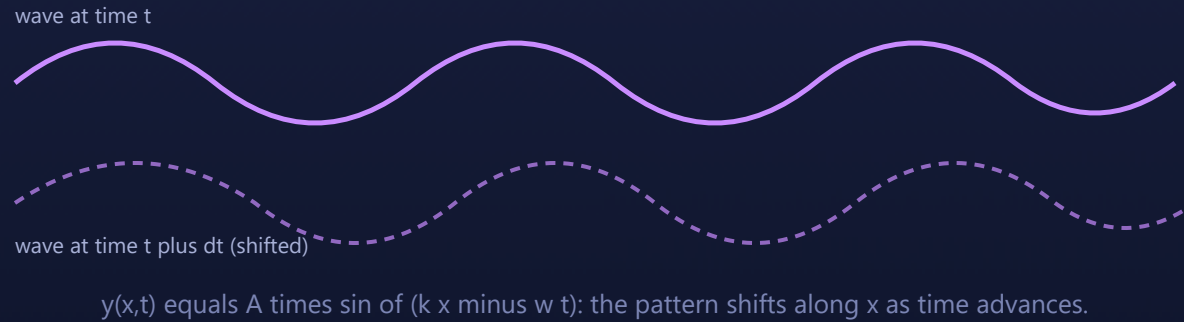


y equals A times $\sin$ of $(w t \text{ plus } \phi)$: a smooth oscillation of amplitude A and period T .

Wave Motion

Combining position and time gives a travelling wave

When the sine argument depends on both position and time, the whole pattern appears to travel. This is used to animate water surfaces, flags and ripples.



TRAVELLING WAVE

Here k is the spatial frequency (wave number) and w is the angular frequency. The minus sign makes the wave move in the positive x direction over time.

OpenGL Animation Procedures

Double buffering for flicker free frames, plus the idle and timer callbacks and redisplay requests that drive smooth real time motion.

Double Buffering

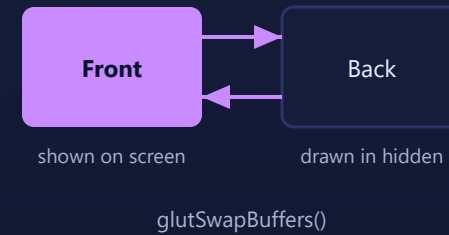
Draw to a hidden buffer, then swap it in to avoid flicker

With a single buffer you would see the picture being erased and redrawn, which flickers. **Double buffering** keeps two buffers: the **front** buffer is shown while the next frame is drawn into the hidden **back** buffer, then the two are swapped.

```
// request a double buffered window
glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB);

// after drawing a frame, swap buffers
glutSwapBuffers();
```

- `GLUT_DOUBLE` asks GLUT for two colour buffers.
- `glutSwapBuffers()` replaces `glFlush()` at the end of the display function.



The finished back buffer is swapped to the front in one step, so no partial frame is seen.

Animation Callbacks

GLUT functions that keep generating new frames



glutIdleFunc

Registers a function that runs whenever the system is idle, used to advance the animation.



glutTimerFunc

Calls a function once after a set number of milliseconds, giving a controlled frame rate.



glutPostRedisplay

Marks the window as needing a redraw, so the display callback runs again.



glutSwapBuffers

Swaps the back and front buffers at the end of each frame.

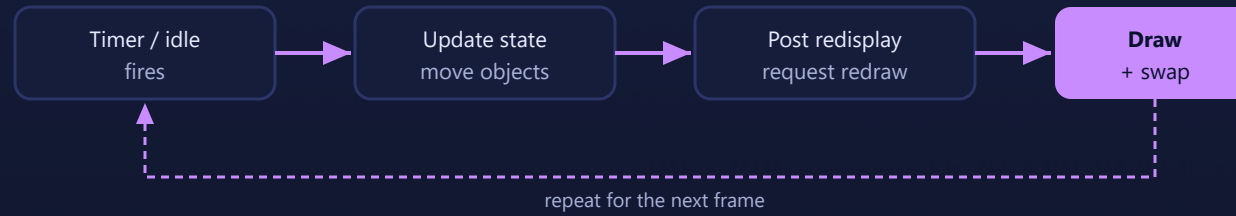
Function	Role in the loop
<code>glutIdleFunc(fn)</code>	Continuously update state when nothing else is happening
<code>glutTimerFunc(ms, fn, v)</code>	Update state at fixed time intervals for a steady frame rate
<code>glutPostRedisplay()</code>	Request that the display callback be called again

IDLE VS TIMER

`glutIdleFunc` runs as fast as possible, so speed depends on the machine. `glutTimerFunc` gives a predictable frame rate that is the same on any machine.

The Animation Loop

How the pieces fit together each frame



Each frame: the timer fires, state updates, a redraw is requested, the scene is drawn and buffers swap.

Complete Example: Moving Object

A ball that moves across the window and bounces off the edges

```
#include <GL/glut.h>

float x = 0.0, y = 0.0;      // ball position
float dx = 0.02, dy = 0.015; // step per frame

void display(void) {
    glClear(GL_COLOR_BUFFER_BIT);
    glColor3f(0.2, 0.6, 1.0);
    glBegin(GL_POLYGON);      // draw a square ball
        glVertex2f(x - 0.05, y - 0.05);
        glVertex2f(x + 0.05, y - 0.05);
        glVertex2f(x + 0.05, y + 0.05);
        glVertex2f(x - 0.05, y + 0.05);
    glEnd();
    glutSwapBuffers();        // double buffering
}

void update(int value) {
    x += dx; y += dy;         // advance position
    if (x > 0.95 || x < -0.95) dx = -dx; // bounce in x
    if (y > 0.95 || y < -0.95) dy = -dy; // bounce in y
    glutPostRedisplay();      // ask for a redraw
    glutTimerFunc(16, update, 0); // ~60 fps
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB);
    glutInitWindowSize(400, 400);
    glutCreateWindow("Bouncing Ball");
    glClearColor(1.0, 1.0, 1.0, 0.0);
    glutDisplayFunc(display);
    glutTimerFunc(16, update, 0); // start the timer loop
    glutMainLoop();
    return 0;
}
```

Alternative: `glutIdleFunc`

The same animation driven by the idle callback instead of a timer

```
void idle(void) {
    x += dx; y += dy;
    if (x > 0.95 || x < -0.95) dx = -dx;
    if (y > 0.95 || y < -0.95) dy = -dy;
    glutPostRedisplay();
}

// in main, instead of the timer:
glutIdleFunc(idle);
```

When to use which

- **`glutIdleFunc`** runs continuously, so motion speed depends on the machine and must be corrected with a time step.
- **`glutTimerFunc`** gives a fixed frame rate that behaves the same on fast and slow machines.
- Set `glutIdleFunc(NULL)` to stop the idle animation.

WATCH OUT

An idle loop with no time scaling makes the object move faster on faster hardware. A timer or a delta time keeps speed constant.

Module 5 Summary

Concepts and Techniques

- Design: storyboard, object definitions, key frames, in betweens.
- Traditional: cel, key framing, tweening, squash and stretch, timing.
- Functions: object manipulation, camera motion, in betweens, morphing.
- Languages: key frame, parameterized and scripting systems.

Motion and Programming

- Characters: articulated figures, skeletons, forward and inverse kinematics.
- Periodic motion: sine and cosine for oscillation and wave motion.
- OpenGL: double buffering with GLUT_DOUBLE and glutSwapBuffers.
- Loop: glutIdleFunc or glutTimerFunc plus glutPostRedisplay.

KEY TAKEAWAY

Animation is the timed display of interpolated frames. Plan with key frames, fill with in betweens, and drive real time playback with double buffering and callbacks.

Sources & References

Textbook chapter and official documentation for this module

- Hearn, Baker & Carithers, **Computer Graphics with OpenGL**, 4th ed., Pearson, 2014, **Chapter 11** (Computer Animation).
- Animation and timing callbacks: [GLUT documentation](#) (glutIdleFunc, glutTimerFunc, glutSwapBuffers, glutPostRedisplay).
- Reference: Edward Angel, **Interactive Computer Graphics**, 5th ed., Pearson, 2008.

LECTURER

Mr. Bharath · B24CS53 Computer Graphics.